

WORK BRIEF — AI VIDEO APP ARCHITECTURE

Mục tiêu

Xây dựng app AI tạo video YouTube tự động, trong đó một **Orchestrator AI** điều phối toàn bộ quy trình, giao việc cho nhiều AI chuyên biệt và sử dụng các công cụ/API/web AI để tạo nội dung, hình ảnh, video, giọng nói và render thành video hoàn chỉnh.

Kiến trúc chính:

A2A = AI ↔ AI

MCP = AI ↔ App/Tools

Provider Gateway = App ↔ dịch vụ AI

Job Queue = quản lý tác vụ dài

1. Kiến trúc tổng thể

USER

↓

ORCHESTRATOR AI

GPT / Claude / Gemini

↓

A2A

└─ Research Agent

└─ Script Agent

└─ Director Agent

└─ Media Agent

└─ QC Agent

↓

MCP

↓

AI VIDEO APP

└─ Project Manager
└─ Job Manager / Queue
└─ Provider Gateway
└─ Browser Workers
└─ Media Storage
└─ Timeline Engine
└─ Render Engine

↓

PROVIDERS

└─ Veo
└─ Seedance
└─ Hailuo
└─ Kling
└─ Image AI
└─ TTS
└─ các provider khác

↓

FFmpeg / Editor

↓

FINAL VIDEO

2. Orchestrator AI

Orchestrator là AI quản lý cao nhất.

Nhiệm vụ:

- Nhận yêu cầu người dùng.

- Lập kế hoạch tạo video.
- Chia công việc cho các Agent.
- Theo dõi trạng thái project.
- Chọn Agent phù hợp.
- Gọi công cụ qua MCP.
- Xử lý lỗi.
- Yêu cầu regenerate scene khi cần.
- Quyết định khi video đủ chất lượng để render.

Không để Orchestrator trực tiếp click website.

3. Agent Layer — A2A

Các AI giao tiếp với nhau thông qua kiến trúc A2A.

Research Agent

Nhiệm vụ:

- Research chủ đề.
- YouTube trends.
- Google Trends.
- Reddit.
- News.
- Competitor analysis.
- Keyword analysis.
- Content gap.

Output dạng JSON có cấu trúc.

Ví dụ:

```
{  
  "topic": "Abandoned American Mining Towns",
```

```
"trend_score": 84,  
"competition_score": 61,  
"opportunity_score": 91  
}
```

Script Agent

Nhiệm vụ:

- Viết outline.
- Hook.
- Script hoàn chỉnh.
- Chia chapter.
- Viết narration.
- Tối ưu retention.

Director Agent

Chuyển script thành danh sách scene.

Mỗi scene cần có:

```
{  
  "scene_id": 12,  
  "narration": "...",  
  "visual_prompt": "...",  
  "duration": 8,  
  "camera": "...",  
  "motion": "...",  
  "media_type": "video"  
}
```

Media Agent

Quyết định scene nên dùng:

- AI video.
- AI image.
- Image animation.
- Stock video.
- Infographic.
- Screenshot.
- Text animation.

Sau đó gửi job tới Provider Gateway.

QC Agent

Kiểm tra:

- Hình sai nội dung.
- Video lỗi.
- Artifact.
- Nhân vật không nhất quán.
- Voice/video lệch.
- Subtitle lỗi.
- Black frame.
- Duplicate clip.
- Duration.
- Chất lượng visual.

Nếu scene lỗi:

QC

↓

Director

↓

Media Agent

↓

Regenerate scene

Không render lại toàn bộ project.

4. MCP Layer

MCP là giao diện giữa AI và app.

Expose các tool cấp cao như:

`create_project()`

`get_project()`

`search_topics()`

`create_script()`

`create_scene()`

`generate_image()`

`generate_video()`

`generate_voice()`

`get_job_status()`

`replace_scene()`

`approve_scene()`

`build_timeline()`

`render_video()`

`get_render_status()`

AI chỉ gọi các tool này.

AI không cần biết code nội bộ của app.

5. Provider Gateway

Tạo một lớp thống nhất cho tất cả AI provider.

Interface ví dụ:

```
generate_video(  
    prompt,  
    duration,  
    aspect_ratio,  
    reference_image,  
    quality  
)
```

Provider Gateway tự chọn:

Veo

Seedance

Hailuo

Kling

Runway

Sora

...

Orchestrator không gọi trực tiếp từng provider.

Gateway cần hỗ trợ:

- Provider priority.
- Cost.

- Quality.
- Availability.
- Retry.
- Fallback.

Ví dụ:

Veo lỗi

↓

Seedance

↓

Hailuo

6. API + Web Adapter

Mỗi provider có Adapter riêng.

Ví dụ:

providers/

veo/

api_adapter.py

web_adapter.py

seedance/

api_adapter.py

web_adapter.py

hailuo/

api_adapter.py

web_adapter.py

Ưu tiên:

Official API

↓

Web Adapter

Nếu không có API thì Browser Worker sử dụng:

- Playwright.
- CDP.
- Browser automation.

Không dùng AI để click từng tọa độ màn hình.

Adapter phải tự:

Open website

↓

Login

↓

Upload image

↓

Enter prompt

↓

Select settings

↓

Generate

↓

Monitor generation

↓

Download result

↓

Return file

7. Job Queue

Tất cả tác vụ lâu phải chạy dạng Job.

Ví dụ:

```
{  
  "job_id": "VID_48291",  
  "provider": "veo",  
  "type": "text_to_video",  
  "status": "GENERATING"  
}
```

Status:

QUEUED

SUBMITTING

GENERATING

DOWNLOADING

DONE

FAILED

Agent không đứng chờ provider tạo video.

Agent gọi:

`generate_video()`

Nhận:

`job_id`

Sau đó:

`get_job_status(job_id)`

hoặc nhận event khi job hoàn thành.

8. Event Bus

Các module giao tiếp bằng event.

Ví dụ:

SCRIPT_COMPLETED

SCENE_CREATED

VIDEO_JOB_STARTED

VIDEO_COMPLETED

VIDEO_FAILED

VOICE_COMPLETED

SCENE_QC_FAILED

SCENE_APPROVED

PROJECT_READY_TO_RENDER

RENDER_COMPLETED

Điều này giúp app không phụ thuộc cứng giữa các module.

9. Project Data

Mỗi project nên lưu:

Project

└─ Topic

└─ Research

└─ Script

└─ Chapters

└─ Scenes

└─ Images

└─ Videos

└─ Voice

└─ Music

└─ Subtitles

└─ Timeline

└─ Jobs

└─ Final Render

Mỗi Scene là entity độc lập.

Ví dụ:

```
{  
  "scene_id": 34,  
  "status": "APPROVED",  
  "voice_file": "034.wav",  
  "video_file": "034.mp4",  
  "duration": 7.4  
}
```

Nhờ vậy có thể regenerate riêng từng scene.

10. Timeline Engine

AI không trực tiếp dựng video.

Director Agent tạo timeline JSON.

Ví dụ:

```
{  
  "scene_id": 34,  
  "video": "034.mp4",  
  "voice": "034.wav",  
  "start": 124.2,  
  "duration": 7.4,  
  "transition": "crossfade"  
}
```

Timeline Engine chuyển JSON thành lệnh render.

11. Render Engine

Render bằng:

- FFmpeg.
- Hoặc editor/render engine hiện có của app.

Render Engine xử lý:

- Video.
- Image.
- Voice.
- Music.
- Subtitle.
- Transition.

- Effects.
- Audio normalization.

Output:

final_video.mp4

12. Workflow hoàn chỉnh

USER

"Tạo video YouTube 20 phút cho thị trường Mỹ"

↓

ORCHESTRATOR

↓

Research Agent

↓

Chọn Topic

↓

Script Agent

↓

Director Agent

↓

120 Scenes

↓

Media Agent

↓

Provider Gateway

↓

Veo / Seedance / Hailuo / Image AI

↓

Job Queue

↓

Media hoàn thành

↓

Voice Generation

↓

Timeline

↓

QC Agent

↓

Regenerate scene lỗi

↓

Approve

↓

Render

↓

FINAL VIDEO

13. Nguyên tắc kiến trúc bắt buộc

Không hard-code Orchestrator với từng AI provider.

Không để LLM trực tiếp điều khiển UI website.

Không để một request giữ kết nối chờ video generate nhiều phút.

Không render lại toàn bộ project nếu chỉ một scene lỗi.

Không để provider-specific logic nằm trong core app.

Mỗi provider phải có Adapter riêng.

Mỗi Agent phải có nhiệm vụ rõ ràng.

A2A dùng cho Agent ↔ Agent.

MCP dùng cho Agent ↔ App/Tool.

Provider Gateway dùng cho App ↔ AI Provider.

Job Queue dùng cho tác vụ asynchronous/long-running.

14. Cấu trúc module đề xuất

app/

core/

orchestrator/

project_manager/

event_bus/

job_manager/

agents/

research/

writer/

director/

media/

qc/

mcp/

server/

tools/

providers/

gateway/

veo/

seedance/

hailuo/

klimg/

browser/

browser_manager/

workers/

media/

storage/

downloader/

timeline/

timeline_engine/

render/

ffmpeg_engine/

database/

models/

repositories/

api/

internal_api/

ui/

15. Yêu cầu triển khai

Không rewrite toàn bộ app hiện tại ngay lập tức.

Thực hiện theo từng phase.

Phase 1

Xây:

- Job Manager.
- Provider Gateway.
- Provider Adapter interface.
- Event Bus.

Phase 2

Chuyển Veo/Seedance/Hailuo hiện tại sang Provider Adapter.

Phase 3

Xây MCP Server cho app.

Phase 4

Xây Orchestrator.

Phase 5

Tách:

- Research Agent.
- Script Agent.
- Director Agent.
- QC Agent.

Phase 6

Thêm A2A communication.

Phase 7

Thêm automatic provider routing, retry, fallback và cost optimization.

Kết quả cuối cùng

Người dùng chỉ cần nhập một yêu cầu cấp cao:

“Tạo cho tôi một video YouTube 20 phút về chủ đề đang được người Mỹ quan tâm.”

Hệ thống phải có khả năng tự động:

Research → chọn topic → viết script → chia scene → tạo prompt → tạo image/video → tạo voice → QC → sửa scene lỗi → dựng timeline → render video hoàn chỉnh.

Thiết kế hệ thống theo hướng modular để sau này có thể thay GPT, Claude, Gemini, Kimi, Veo, Seedance hoặc bất kỳ provider nào mà không phải sửa toàn bộ app.